

Pair-PM — Build Spec

Generative UI Global Hackathon — May 9, 2026

1. The pitch (read this first)

Pair-PM is a senior PM that lives inside your spec doc. As you write a PRD, it draws the user-flow diagram beside you, comments on ambiguities and missing edge cases in real time, and hands you an engineering-ready spec card when you're done.

One-liner: *Not a chatbot you ask. A reviewer who's already in the doc.*

Why it wins the hackathon: passes "couldn't be a chatbot" structurally — the agent reads as you type, draws what's implicit in the prose, and asks what's missing. Three live surfaces, side by side, all updating together. The "agent is drawing while I'm typing" moment is the wow.

2. Screen layout

Three zones, all visible simultaneously:

LEFT (50% width)	RIGHT-TOP (~60% height)
PRD Editor	Live Flow Diagram
(Notion-style)	(A2UI canvas)
PM types here	Agent draws nodes/edges as PM types
	RIGHT-BOTTOM (~40%)
	Reviewer Comments
	(stacked cards)
	Agent's questions on ambiguities and gaps

A small **action bar** above the canvas: “Generate engineering handoff” button + a “resolved 0/0” counter.

3. The 90-second demo (locked)

Frame 1 — 0:00 (the opening)

- **LEFT:** Blank page titled “PRD: Date-range filtering for analytics dashboard”
- **RIGHT-TOP:** Empty canvas, faint label “Flow diagram (waiting for content...)”
- **RIGHT-BOTTOM:** Empty, faint label “Reviewer comments will appear here”
- **You say:** “PMs write specs. Engineers read them and immediately ask three questions the spec didn’t answer. Today the PM finds out in standup. We wanted them to find out as they type.”

Frame 2 — 0:15 (first paragraph types in)

- **LEFT:** PM types: “Users on the analytics dashboard need to filter their data by a custom date range. Today, they can only choose preset windows (7, 30, 90 days). The new flow lets them pick a start date and end date and re-run the query.”
- **RIGHT-TOP:** A 5-node flow appears, animated left-to-right, ~200ms between nodes:

[User on dashboard] → [Opens filter] → [Picks start date] → [Picks end date] -

- **THIS IS THE WOW MOMENT.** Audience sees the agent drawing the flow of what was just typed.

Frame 3 — 0:35 (second paragraph + first comment)

- **LEFT:** PM adds: “Users should also be able to group results by region. The combination of date range and region grouping should update the chart in real time.”
- **RIGHT-TOP:** Diagram grows additively — new branch sprouts off existing nodes for region grouping. **Existing nodes stay put.**
- **RIGHT-BOTTOM:** First comment card fades in:

 **Pair-PM Quoted:** “filter their data by a custom date range” **What’s the max range allowed?** 30 days, 1 year, all-time? And is the picker a single calendar or two separate pickers? [Mark resolved]

Frame 4 — 0:55 (PM addresses comment 1, comment 2 appears)

- **LEFT:** PM adds: *"Date range is capped at 1 year. Use a two-pane calendar picker (start + end)."*
- **RIGHT-BOTTOM:** Comment 1's border turns green, slides up to a resolved row. Counter: *"1 of 1 resolved."* Then comment 2 fades in:

... **Pair-PM Quoted:** *"group results by region"* **What if a region has no data in the selected range?** The flow doesn't handle empty state — hide the region, show a placeholder, or fall back to a wider window? [Mark resolved]

- **RIGHT-TOP:** The "Selects region grouping?" node briefly pulses yellow, tying the comment to the node it concerns.

Frame 5 — 1:15 (PM addresses comment 2)

- **LEFT:** PM adds: *"Empty state: regions with zero data in the range show a 'no data' placeholder, not hidden."*
- **RIGHT-TOP:** Diagram grows — the yellow node sprouts a new branch for empty state. Yellow clears.
- **RIGHT-BOTTOM:** Comment 2 turns green. Counter: *"2 of 2 resolved."*

Frame 6 — 1:30 (the handoff)

- PM clicks **"Generate engineering handoff."**
- **RIGHT-TOP:** Flow diagram is replaced (slide animation) by a **Spec Card** containing:
 - Cleaned-up PRD text
 - Embedded mini-flow diagram
 - Resolved questions list ("Max range: 1 year ✓ | Empty state: placeholder ✓")
 - Auto-extracted acceptance criteria in Given/When/Then format (3–4 of them)
 - [Copy as Linear ticket] button

Frame 7 — 1:45 (closer)

"PMs don't have a writing problem. They have a peer-review-while-writing problem. We made the senior PM available to every PM — watching every doc, drawing what's implicit, asking what's missing. Same six minutes of writing. Engineering-ready output."

The 8 seconds that win the room: 0:15–0:23 — the first 5 nodes drawing themselves

into existence.

4. Data model (what the agent emits)

The agent maintains and emits structured state. Frontend diffs against previous state to animate additively.

```
interface PairPMState {
  flowDiagram: {
    nodes: Array<{
      id: string;           // stable id – used for diffing
      label: string;        // "Picks start date"
      type: "action" | "decision" | "endpoint" | "empty_state";
      sourceQuote: string;   // the prose snippet that justified this node
      highlight?: "yellow" | "green" | null;
    }>;
    edges: Array<{
      from: string;         // node id
      to: string;           // node id
      label?: string;       // "yes" / "no" for decision branches
    }>;
  };
  comments: Array<{
    id: string;
    quote: string;         // exact prose excerpt the comment refers to
    question: string;      // the agent's question
    relatedNodeId?: string; // ties to a flow node
    status: "open" | "resolved";
  }>;
  specCard?: {             // populated when "Generate handoff" is clicked
    refinedPRD: string;
    acceptanceCriteria: string[];
    resolvedQuestions: Array<{ q: string; a: string }>;
  };
}
```

Key invariant: node IDs are *stable across updates*. New paragraphs add new nodes; existing nodes never get recreated. This is what makes the diagram feel alive.

5. Agent prompt (drop-in)

Use this in `apps/agent/src/runtime.py`, replacing the leads-related prompt.

You are Pair-PM, a senior product manager reviewing a teammate's PRD as they write

You have three responsibilities:

1. FLOW DIAGRAM — Maintain a live diagram of the user journey described in the PRD
2. REVIEWER COMMENTS — Identify ambiguities, missing edge cases, undefined person
 - Quote the exact prose it refers to
 - Ask one specific, actionable question
 - Tie to a flow node when possible
3. SPEC CARD — When asked to generate the engineering handoff, output:
 - A cleaned, unambiguous version of the PRD
 - 3-5 acceptance criteria in Given/When/Then format
 - A list of resolved questions with answers from the doc

Comment style: senior, calm, specific. Never vague ("consider edge cases"). Always

Output schema: JSON matching PairPMState (provided separately). Emit the FULL state

When the doc is empty or barely-started, emit empty arrays, not placeholder content

6. Components to build

A. Flow Diagram (A2UI declarative) — *the hero, build first*

- Renders nodes as labeled rectangles, edges as arrows
- Layout: simple horizontal/branching tree (don't build a force-directed engine — overkill for 3 hrs)
- Animation: new nodes fade + slide in, ~200ms stagger
- Highlight states: yellow pulse (linked to a comment), green flash (newly added)
- **Library option:** use ReactFlow (`reactflow` npm) with custom node component. Saves 90% of the work.

B. Comment Card (controlled component, `useComponent`)

- Quote block at top
- Question text

- “Mark resolved” button
- States: open (default border), resolved (green border, 60% opacity, moved to resolved row)
- Optional `relatedNodeId` triggers a yellow pulse on the linked node when card is hovered

C. Spec Card (controlled component)

- Section: refined PRD (markdown rendered)
- Section: embedded mini flow diagram (read-only)
- Section: acceptance criteria (Given/When/Then format, monospace)
- Section: resolved questions (Q/A pairs)
- “Copy as Linear ticket” button (formats as Markdown, copies to clipboard)

D. PRD Editor (left pane)

- Use **TipTap** with a minimal config (heading, paragraph, list)
- On every blur or every 2-second debounce, push the doc content to the agent
- Don’t build live-typing streaming — debounced updates are smoother for the demo and easier to build

7. Three-hour build plan

Hour 1 — Plumbing

- **Person 1 (kit setup):** clone starter kit, get it running locally, replace the leads prompt with the Pair-PM prompt
- **Person 2 (PRD editor):** drop in TipTap on the left pane; wire debounced content push to the agent
- **Person 3 (flow diagram):** install ReactFlow; render a hardcoded 3-node diagram on the right-top to verify the rendering pipeline works
- **Person 4 (data + script):** seed the demo PRD content; write the exact paragraph progression for the demo; rehearse the typing cadence

Hour 2 — The two hero components

- **Person 1:** wire agent output → flow diagram state with stable-ID diffing (the make-or-break correctness piece)
- **Person 2 + 3:** build the Comment Card + comment list pane; wire to agent output
- **Person 4:** build the Spec Card component (can use mocked content first, real content if time permits)

Hour 3 — Polish + record

- **Everyone:** end-to-end rehearsal. Find the rough edges. Tune timing.
 - **Person 4:** record the fallback video while others polish (do this BEFORE final tweaks so you have *something* to submit if everything breaks)
 - **Person 1:** final demo recording for submission
 - **Person 2:** write submission text + GitHub README
-

8. Stack

- **Framework:** Next.js (from starter kit)
 - **Editor:** TipTap
 - **Diagram:** ReactFlow with custom node component
 - **Agent surface:** CopilotKit with `useComponent` for comment + spec cards
 - **Streaming:** A2UI for the flow diagram nodes/edges
 - **Agent runtime:** LangChain Deep Agents (from kit) with Gemini 3.1 Flash-Lite (or swap to Claude Sonnet if your team prefers — one-line edit per kit docs)
 - **Persistence:** kit's built-in Postgres threads
 - **MCP:** can stay disabled for the demo — the PRD lives in the editor, not Notion. (If time permits in hour 3, add Notion MCP to *load* a PRD, but it's optional polish.)
-

9. Risks + mitigations

Risk 1: Diagram redraws from scratch on every update (flickers, feels cheap)

Mitigation: stable node IDs in the agent's output schema. Frontend diffs by ID — only renders changes. Test this in hour 1, not hour 3.

Risk 2: Comment timing feels canned

Mitigation: add a 1.5-second delay between when the agent decides to comment and when the card appears. Looks like the reviewer is "thinking." Critical perceived-quality detail.

Risk 3: Live demo breaks on stage

Mitigation: record the fallback video at hour 3 start, not hour 3 end. Worst case, you play the video and pitch over it. Judges accept this.

Risk 4: Agent emits malformed JSON (state breaks)

Mitigation: wrap the agent output in a JSON-schema validator. On parse failure, retain previous state. Don't crash the UI mid-demo.

Risk 5: Pre-scripted feel

Mitigation: type *the same paragraphs you tested with*. Don't ad-lib on stage. The agent's reactions should feel emergent but be reliably triggered. Plant the surprises, rehearse the beats.

10. Submission text

Pair-PM — *A senior PM that lives inside your spec doc.*

As you write a PRD, Pair-PM draws the user-flow diagram beside you, comments on ambiguities and missing edge cases in real time, and hands you an engineering-ready spec card when you're done. The agent reads as you type — drawing what's implicit in the prose and asking what's missing.

Demo: A PM types a PRD for date-range filtering. As paragraphs land, a flow diagram grows additively on the right. Comments appear on ambiguities ("max date range?") and missing edge cases ("empty state?"). PM addresses each in the doc; comments resolve, diagram grows. One click generates an engineering handoff card with refined PRD, acceptance criteria, and resolved questions.

Why this isn't a chatbot: the agent never waits for a question. It watches as you write, draws what your prose implies, and surfaces gaps as comments tied to specific quotes. The interaction is pair-editing, not chat.

Protocols used: A2UI (flow diagram), CopilotKit (`useComponent` for cards), MCP-ready (Notion integration optional).

11. The pitch sentence to land at the end

"Today, the senior PM is one person on your team. Tomorrow, they're in every doc, on every team, watching every spec. Same six minutes of writing — engineering-ready output."

12. Open decisions (lock before starting)

1. **Notion MCP yes/no for hour 3?** Default no. Adds polish but eats time.
 2. **Gemini or Claude for the agent?** Default Gemini (kit default). Switch only if your team has strong Claude prompting muscle.
 3. **Recorded fallback at hour 2:30 or hour 3:00?** Default 2:30. Earlier is safer.
 4. **Live typing or pre-recorded typing in the demo?** Default live typing — the "agent reacts to me" moment is dramatically stronger when it's clearly real-time. But have the recording ready.
-

Last edited: May 9, 2026. Built for the Generative UI Global Hackathon. MIT.